

Unit 1: Basics: hyperparameter tuning and scaling

Lecture notes (read before the handout)

dl-alchemy (after Stanford CS 312: Deep Learning Alchemy)

2026-09-19, v1.1

Reconstruction: CS 312 had published no Unit 1 material on this date. Citations checked against arXiv.

1 What this unit asks, and why anyone cares

You have a fixed amount of compute and a transformer to pre-train. Before any clever idea can be tested, two plain questions have to be answered well.

1. **Tuning.** Which learning rate, batch size, warmup, decay schedule and weight decay? How sharp is the optimum, and what does it cost to miss it?
2. **Scaling.** How should the compute be split between a bigger model and more tokens, and how far can a fit made on small runs be trusted when extrapolated?

They matter more than they look. A large run is launched once, so its hyperparameters are **predicted** from small runs, not tuned. And almost every comparison you will make in units 2 to 7 (method A vs method B) is only as good as the tuning of both arms. Two published cautionary tales, both checked below: the 2020 and 2022 scaling laws disagreed about how to spend compute mostly because of tuning and accounting details ([Section 3](#), Porian et al.), and a literature-wide disagreement about whether large batches hurt was “largely explained by differences in metaparameter tuning and compute budgets” (Shallue et al.).

The quiz will show you a code or config diff against the baseline and ask what happens to the final validation loss. [Section 2](#) gives four pictures to reason with, [Section 3](#) the numbers other people measured, [Section 4](#) three worked predictions, and [Section 8](#) practice questions.

How to answer a prediction question (use this every time)

1. **What changed, and what was silently held fixed?** Tokens or steps? Was the learning rate re-tuned for the new arm or kept? Did the schedule length change with the run length?
2. **Which term of which mental model moves?** Name it: the noise floor, the unconverged flat directions, the stability edge, the parameter term, the data term.
3. **Sign.** Better, worse, or inside the noise floor?
4. **Size.** Compare with anchors you know: the seed-to-seed spread, what doubling the tokens buys, what a $3\times$ wrong learning rate costs. You will measure these anchors yourself; they are the most useful page of your report.
5. **Confidence**, and what result would make you drop the model you used.

2 Four mental models

2.1 The noisy quadratic: why the loss-vs-learning-rate curve is a bowl with a cliff

Take one direction of parameter space with curvature h , and SGD with learning rate η on minibatches of size B . The gradient is hx plus noise of variance σ^2/B . One step is $x_{t+1} = (1 - \eta h)x_t - \eta \xi_t$, so the expected loss $\frac{1}{2}hx^2$ after T steps is

$$\mathbb{E}[L_T] - L^* \approx \underbrace{\frac{1}{2}hx_0^2 e^{-2\eta h T}}_{\text{not yet converged}} + \underbrace{\frac{\eta \sigma^2}{4B}}_{\text{noise floor}}, \quad \text{valid for } \eta h \ll 2. \quad (1)$$

A real loss has many directions with curvatures from large (stiff) to tiny (flat). Each has its own copy of Equation 1, and the step is stable only if $\eta < 2/h_{\max}$. Read the three regimes off the formula:

- **Learning rate too low** (left side of the bowl). The flat directions, where $\eta h T \lesssim 1$, have not converged. Loss rises gently as η falls, roughly linearly in $\log \eta$.
- **Learning rate high but stable** (right side). Everything has converged to its noise floor, which grows in proportion to η .
- **Too high**. Past $2/h_{\max}$ the stiffest direction diverges. This is a cliff, not a slope. Gradient clipping, Adam’s normalization and warmup blunt the cliff and move it, but the asymmetry survives: **missing low is cheap, missing high is expensive**.

Mental model: Travel fast, then cool down

The two terms of Equation 1 want opposite things: travel along flat directions wants a high η for a long time; a low noise floor wants a small η . A decay schedule gets both. When η drops, the noise term falls to its new floor within about $1/(\eta h)$ steps per direction, while the distance already travelled is kept. Three consequences: (i) loss falls **suddenly** during the decay phase; (ii) a constant-learning-rate run looks worse mid-training than it really is, because its progress is hidden under noise; (iii) the model predicts the size of the drop grows with η_{peak}/B . Point (iii) is this model’s prediction, not a published number. It is yours to test.

Minimizing Equation 1 over a constant η gives $\eta^* \approx \ln(\dots)/(2hT)$: **longer runs want lower learning rates**. Bjorck et al. (2024) measured exactly this for LLMs: “longer training necessitates smaller LR”, following a power law in the token horizon. Adam is not SGD, so treat Equation 1 as a picture for signs and trends, not for decimals.

2.2 Gradient noise scale: what batch size buys

In Equation 1 the noise floor depends on η/B only. So for SGD, multiplying B and η by the same factor leaves the noise alone and cuts the number of steps: **perfect scaling**. It ends when η reaches the stability edge and cannot grow further; after that, extra batch is wasted. McCandlish et al. (2018) make this quantitative with the gradient noise scale:

$$B_{\text{simple}} = \frac{\text{tr}(\Sigma)}{|G|^2}, \quad \eta_{\text{opt}}(B) = \frac{\eta_{\max}}{1 + B_{\text{noise}}/B}, \quad \left(\frac{S}{S_{\min}} - 1\right)\left(\frac{E}{E_{\min}} - 1\right) = 1, \quad (2)$$

where Σ is the per-example gradient covariance, G the true gradient, S optimizer steps and $E = BS$ examples processed to reach a target loss. Define $B_{\text{crit}} = E_{\min}/S_{\min}$. Then $S/S_{\min} = 1 + B_{\text{crit}}/B$ and

$E/E_{\min} = 1 + B/B_{\text{crit}}$: below B_{crit} you pay in steps (time), above it you pay in examples (compute). At $B = B_{\text{crit}}$ you pay $2\times$ of both.

Two facts from the same paper to keep: the noise scale **grows during training** (the gradient shrinks while the noise does not), and at a fixed loss it barely depends on model size. For Adam the learning-rate rule is softer than linear: they report $\eta \propto B^a$ with a between 0.5 and 1 before it flattens; Malladi et al. (2022) derive $a = \frac{1}{2}$ from an SDE argument (Section 3).

Note: Fixed tokens or fixed steps? In this course runs are compared at a fixed token budget. Then a $4\times$ larger batch means $4\times$ fewer steps. Below B_{crit} that is nearly free once η is re-tuned. Above B_{crit} it costs loss. At fixed **steps**, larger batch always helps. Many confusions are just this.

2.3 Warmup: buying a higher stable learning rate

At initialization the loss surface is poorly conditioned and Adam’s second-moment estimate is built from very few gradients. A large first step can land somewhere worse. Kalra and Barkeshli (2024) find that the main benefit of warmup is “allowing the network to tolerate larger” target learning rates “by forcing the network to more well-conditioned areas of the loss landscape”. In the language of Section 2.1: warmup moves the cliff to the right. It follows that warmup should matter little at a conservative learning rate and a lot near the edge. Wortsman et al. (2023) see the matching effect at scale: longer warmup reduces learning-rate sensitivity, more so for larger models.

2.4 Two bottlenecks: the scaling-law picture

Hoffmann et al. (2022) model the final loss of a model with N parameters trained on D tokens as

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}, \quad C \approx 6ND. \quad (3)$$

E is the entropy of the data (irreducible). The second term is what a finite model cannot represent; the third is what finite data cannot teach. With compute C fixed, $D = C/6N$ and the optimum is

$$N_{\text{opt}} = G \left(\frac{C}{6} \right)^{\frac{\beta}{\alpha+\beta}}, \quad D_{\text{opt}} = G^{-1} \left(\frac{C}{6} \right)^{\frac{\alpha}{\alpha+\beta}}, \quad G = \left(\frac{\alpha A}{\beta B} \right)^{\frac{1}{\alpha+\beta}}. \quad (4)$$

Mental model: An IsoFLOP curve is a shallow bowl

At the optimum, $\alpha AN^{-\alpha} = \beta BD^{-\beta}$: the two reducible terms are comparable (ratio $\beta : \alpha$), so neither bottleneck dominates. Move away at fixed compute and one term falls while the other rises, which makes loss vs $\log N$ a shallow bowl. Being $2\times$ off in model size is cheap; $4\times$ is not (numbers in Section 4). Because the bowl is shallow, **locating** its minimum from noisy runs is hard, and small errors in the minimum become large errors in the fitted exponents.

$C \approx 6ND$ is itself a model. It ignores attention FLOPs (for us roughly $T/12d \approx 17\%$ extra at $T = 512$, $d = 256$) and hides a choice: does N include the embedding and output matrices? At our scale those matrices together are two thirds the size of the transformer body, so the choice changes every number. This is one of the three reasons the two famous scaling laws disagreed (Section 3).

3 What is already known

All entries were checked against the papers on 2026-09-19. Absolute losses depend on dataset and tokenizer and do not transfer to our setup; exponents, ratios and shapes are the useful part.

Source	Finding	Scale / caveat
Kaplan et al. 2020	$L(N) \propto N^{-0.076}$, $L(D) \propto D^{-0.095}$. Compute-optimal: $N \propto C^{0.73}$, $B \propto C^{0.24}$, steps $\propto C^{0.03}$: spend almost everything on model size. $B_{\text{crit}}(L) = B_*/L^{1/0.21}$, $B_* \approx 2 \times 10^8$ tokens. “Performance depends very weakly on ... depth vs. width”.	N excludes embeddings; fixed 3000-step warmup. Superseded on allocation.
Hoffmann et al. 2022 (Chinchilla)	Three methods give $N_{\text{opt}} \propto C^a$ with $a = 0.50, 0.49, 0.46$: scale model and data about equally, ≈ 20 tokens per parameter (70B on 1.4T). Parametric fit: $E = 1.69$, $A = 406.4$, $B = 410.7$, $\alpha = 0.34$, $\beta = 0.28$. Cosine cycle should match the run length: overshooting by more than 25% “leads to clear drops in performance”; decay is $10\times$.	400+ models, 70M to 16B.
Besiroglu et al. 2024	Refit of Hoffmann’s parametric method from the published plots: $E = 1.82$, $A = 482$, $B = 2085$, $\alpha = 0.35$, $\beta = 0.37$, giving $a = 0.51$ (≈ 20 tokens/param). The original values imply ≈ 70 tokens/param at Chinchilla scale, inconsistent with the paper’s own other two methods; the reported confidence intervals were implausibly narrow.	Lesson: fits of Equation 3 are fragile.
Porian et al. 2024	Reproduced Kaplan’s exponent ($a = 0.86$), then removed the gap to Hoffmann step by step: count last-layer FLOPs ($a = 0.70$), stop using a fixed long warmup that swamps small models (0.60), tune learning rate and batch size per scale (≈ 0.50). Also: AdamW $\beta_2 = 0.95$ is suboptimal at small batch, 0.99 is better; learning-rate decay is not needed for the exponent (it still lowers the loss). Optimal LR $\propto N^{-0.11}$, optimal batch $\propto N^{0.21}$ (approx.).	Models up to 901M; two datasets.
Bi et al. 2024 (DeepSeek LLM)	Near-optimal $\eta = 0.3118C^{-0.125}$ and $B = 0.292C^{0.3271}$: batch grows and LR falls slowly with compute. The near-optimal region is wide: many (η, B) pairs are within 0.25% of the best loss.	Their data and architecture; the constants are not portable.
Bjorck et al. 2024	At fixed model size the optimal LR falls as the token horizon grows, following a power law, so it can be extrapolated from short runs.	LLM pre-training.
Hägele et al. 2024	Constant LR plus a cooldown (WSD) “scales predictably and reliably similar to cosine”. A cooldown of 20% of the steps matches cosine (shorter is enough for long runs); the shape $1 - \sqrt{t}$ (t = progress through the cooldown) beats linear; the best LR is about half of cosine’s best peak LR; weight averaging helps mid-run but does not reach an explicit cooldown.	33M to 8B models.

McCandlish et al. 2018	Equation 2 . The noise scale predicts the largest useful batch across many domains, grows during training, and depends on model size mainly through the loss reached.	Mostly SGD; Adam rule is softer.
Malladi et al. 2022	Square-root rule for Adam. For $B \rightarrow \kappa B$: $\eta \rightarrow \eta\sqrt{\kappa}$, $1 - \beta_{1,2} \rightarrow \kappa(1 - \beta_{1,2})$, $\varepsilon \rightarrow \varepsilon/\sqrt{\kappa}$.	SDE limit; small η .
Marek et al. 2025	Small batches train stably down to batch size one and are more robust to hyperparameters, if β_2 is scaled to keep its half-life fixed in tokens. They advise against gradient accumulation on a single device.	Agrees with Porian on β_2 .
Zhang et al. 2024	Critical batch size “scales primarily with data size rather than model size”.	85M to 1.2B.
Wortsman et al. 2023	Large-scale instabilities (attention-logit growth, output-logit divergence) reproduce in small models at high LR; fixes are qk-layernorm and z-loss (10^{-4}). Defines LR sensitivity : the mean excess loss over a 3-decade LR range. Longer warmup and independent weight decay reduce it; without qk-layernorm the diverging LR shrinks as models grow.	$\approx 10\text{M}$ to 4.8B . Our scale is inside their range.
Shallue et al. 2018	Steps-to-target vs batch size: perfect scaling, then diminishing returns, then none; the thresholds vary enormously across workloads. No evidence that large batches hurt generalization once everything is re-tuned.	168k models, 35 workloads.

Table 1: Published results to anchor predictions. Full references in [Section 9](#).

Reading list. Read: Hoffmann et al. §3 (the three fitting methods) and Porian et al. §1 to §3 (how small details bend a scaling law; the closest thing to this unit in paper form). **Skim:** McCandlish et al. §2; Hägele et al. §3 to §4; Wortsman et al. §3. **Optional:** the rest of [Table 1](#).

4 Worked predictions

Each walkthrough uses the five steps of [Section 1](#), then checks against a published result.

Example (cosine-too-long): The schedule thinks the run is twice as long as it is

```
- sched = cosine(peak_lr, total_steps=train_steps, final_frac=0.1)
+ sched = cosine(peak_lr, total_steps=2 * train_steps, final_frac=0.1)
```

Changed: only the schedule. Tokens, peak LR and everything else fixed. **Term that moves:** the noise floor of [Equation 1](#). The run now stops halfway down the cosine, at $0.1 + 0.9 \times \frac{1}{2} = 0.55$ of peak instead of 0.1, so the noise term at the end is several times larger; travel along flat directions is slightly **better** (higher average LR). **Sign:** worse, since the last steps of annealing are where loss

falls fastest. **Size:** well outside seed noise. **Published:** Hoffmann et al.: overshooting the cycle length by more than 25% “leads to clear drops in performance”. **What would change the answer:** a tiny peak LR, where the noise floor is negligible and the left-side term dominates.

Example (wsd-swap): Replace cosine by constant + 20% cooldown, same peak LR

Changed: schedule shape; both end annealed. **Terms:** WSD spends longer at peak LR (more travel) and less time annealing (the floor is reached a bit less completely). The effects oppose each other. **Sign and size:** close to a tie; call it “=” unless your noise floor is very tight. **Published:** Hägele et al.: a 20% cooldown matches cosine, a $1 - \sqrt{t}$ cooldown beats a linear one, and the best WSD learning rate is about half of cosine’s best peak. So at a peak LR tuned for cosine, WSD is if anything slightly disadvantaged; re-tuned, it matches. **Notice the trap:** “same peak LR” is not “each arm tuned”.

Example (model-too-big): Same compute, model $2\times$ too large (so half the tokens)

Plug into [Equation 3](#) with $D = C/6N$ and compare with the optimum (computed for these notes from the two published fits):

Fit	C (FLOPs)	$N = 2N_{\text{opt}}$	$N = 4N_{\text{opt}}$	doubling C at the optimum
Hoffmann	10^{18}	+0.042	+0.167	−0.186
Besiroglu	10^{18}	+0.052	+0.211	−0.195
Hoffmann	10^{17}	+0.060	+0.237	
Besiroglu	10^{17}	+0.078	+0.317	

Reading: a $2\times$ sizing error costs about a quarter of what doubling the compute buys; a $4\times$ error costs about as much as throwing away half the compute. Too small by $2\times$ or $4\times$ costs about the same as too large (the bowl is roughly symmetric in $\log N$ for these fits). Penalties in nats **shrink** as compute grows, because the whole reducible loss shrinks. **Caveat:** these fits come from another dataset and tokenizer, and at 10^{16} to 10^{17} FLOPs (our range) they are extrapolated below the data they were fit on. Trust the ratios, not the nats.

5 Where the literature disagrees or is thin

These are the places where your own runs tell you something a paper cannot.

- **Which N , which C ?** With or without embeddings and the output layer, with or without attention FLOPs. At our scale the answer changes the fitted exponent (Porian’s largest single correction, $0.86 \rightarrow 0.70$).
- **Are the exponents even stable?** Hoffmann’s own three methods differ (0.46 to 0.50), and the parametric one did not survive a refit. What does a fit from three small budgets predict one budget up, and how wrong is it?
- **What sets the critical batch size?** The loss reached (McCandlish, Kaplan) or the amount of data (Zhang)? Neither tells you where B_{crit} is for a 6M-parameter model on 120M tokens.

- **How does the best LR move?** Down with model size (Porian, Wortsman), down with run length (Bjorck), up with batch size (by $\sqrt{\kappa}$? linearly?). The published exponents are small, so over our narrow range the shifts may be below one grid step of $3\times$. Whether they are visible is an empirical question.
- **Does β_2 matter?** Porian and Marek say yes at small batch. Most recipes never touch it.
- **Is warmup needed at all at this scale,** or only near the stability edge?

6 Links to other units

Unit 1 has no predecessor. It feeds the rest: Unit 2 (hyperparameter invariants) turns the trends of [Section 5](#) into exact rules for moving hyperparameters together; Unit 3 (sharp and flat basins) replaces the noisy quadratic by a richer landscape picture of the cooldown drop. Your anchor table and your measured noise floor are reused by every later quiz.

7 Experiment design for this unit

- **Say what is fixed.** Tokens, steps, or compute. Batch-size and model-size experiments reverse their conclusion depending on this ([Section 2.2](#)).
- **Tune each arm.** A comparison at one shared LR measures how well that LR suits each arm. CS 312's protocol: best loss over the grid $\{10^{-4}, 3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}, 3 \times 10^{-2}\}$. If the best LR sits on the edge of the grid, extend the grid; an edge minimum is not a minimum.
- **Know the noise floor.** Differences under about twice the seed-to-seed standard deviation are “=”.
- Arms that share a seed share the data order, so paired differences are tighter than the seed spread suggests.
- **One knob at a time, and watch the hidden couplings.** Batch size changes the step count. Run length changes the schedule. Width changes parameters, FLOPs and the best LR at once. Warmup given as a fraction of steps changes when the step count changes.
- **Plot against log of the knob,** keep diverged runs on the plot (they locate the cliff), and look at the whole loss curve, not only the final number: [Section 2.1](#) says a mid-run comparison between schedules misleads.
- **Short proxy runs can flip rankings,** because the best LR moves with the horizon (Bjorck). Use them to find the cliff, not to pick winners.
- **For scaling fits:** several model sizes per compute budget so the bowl's minimum is bracketed; fit in log space; hold out the largest budget and predict it **before** running it.

8 Practice questions

Answers follow. They come from the papers or from the formulas above, not from runs on this machine.

1. Compute doubles. By what factor should the model and the token count grow under Hoffmann's IsoFLOP exponents? Under Kaplan's?
2. You train at $B = 4B_{\text{crit}}$. Relative to the best possible, how many more optimizer steps and how many more training examples do you need?
3. A run sits at steady state at constant LR. You halve the LR. In the noisy quadratic, what happens to the excess loss, and on what timescale?
4. Adam, batch size $\times 4$. What does the square-root rule do to η , $\beta_1 = 0.9$, $\beta_2 = 0.95$? What β_2 does the fixed-token-half-life rule give?

5. Of the three corrections in Porian et al., which moved the exponent most, and why should that one worry us in particular?
6. The baseline’s warmup is removed. For which learning rates do you expect (a) no change, (b) a worse loss or divergence?

Answers.

1. Hoffmann: $2^{0.49} \approx 1.40$ for the model and $2^{0.51} \approx 1.42$ for the data, about $\sqrt{2}$ each. Kaplan: $2^{0.73} \approx 1.66$ for the model and $2^{0.27} \approx 1.21$ for the data.
2. $S/S_{\min} = 1 + \frac{1}{4} = 1.25$ and $E/E_{\min} = 1 + 4 = 5$. Large batches buy little time and cost a lot of compute.
3. The noise term $\eta\sigma^2/4B$ halves; the unconverged term does not change at that moment. Each direction relaxes in about $1/(\eta h)$ steps, so stiff directions drop at once and most of the visible drop comes early.
4. $\eta \rightarrow 2\eta$; $\beta_1 \rightarrow 1 - 4(0.1) = 0.6$; $\beta_2 \rightarrow 1 - 4(0.05) = 0.8$; $\varepsilon \rightarrow \varepsilon/2$. Fixed half-life: $\beta_2 \rightarrow 0.95^4 \approx 0.815$. The two rules agree to first order, since $\beta^\kappa \approx 1 - \kappa(1 - \beta)$.
5. Counting last-layer FLOPs ($0.86 \rightarrow 0.70$). In small models the output layer is a large share of compute, so ignoring it makes small models look cheaper than they are. In our baseline the output matrix alone is a third the size of the transformer body.
6. (a) Conservative LR, well left of the cliff: warmup mainly raises the tolerable LR (Kalra and Barkeshli), so nothing much changes. (b) LR near the top of the grid: the cliff moves left, so runs that were fine become worse or diverge (Wortsman: longer warmup lowers LR sensitivity). How far left at our scale is unknown.

9 References

- J. Kaplan et al. (2020). Scaling Laws for Neural Language Models. [arXiv:2001.08361](#)
- J. Hoffmann, S. Borgeaud, A. Mensch et al. (2022). Training Compute-Optimal Large Language Models. [arXiv:2203.15556](#)
- T. Besiroglu, E. Erdil, M. Barnett, J. You (2024). Chinchilla Scaling: A replication attempt. [arXiv:2404.10102](#)
- T. Porian, M. Wortsman, J. Jitsev, L. Schmidt, Y. Carmon (2024). Resolving Discrepancies in Compute-Optimal Scaling of Language Models. [arXiv:2406.19146](#)
- X. Bi et al. (DeepSeek-AI) (2024). DeepSeek LLM: Scaling Open-Source Language Models with Longtermism. [arXiv:2401.02954](#)
- J. Bjorck, A. Benhaim, V. Chaudhary, F. Wei, X. Song (2024). Scaling Optimal LR Across Token Horizons. [arXiv:2409.19913](#)
- A. Hägele, E. Bakouch, A. Kosson, L. Ben Allal, L. Von Werra, M. Jaggi (2024). Scaling Laws and Compute-Optimal Training Beyond Fixed Training Durations. [arXiv:2405.18392](#)
- S. McCandlish, J. Kaplan, D. Amodei, OpenAI Dota Team (2018). An Empirical Model of Large-Batch Training. [arXiv:1812.06162](#)
- S. Malladi, K. Lyu, A. Panigrahi, S. Arora (2022). On the SDEs and Scaling Rules for Adaptive Gradient Algorithms. [arXiv:2205.10287](#)
- M. Marek, S. Lotfi, A. Somasundaram, A. G. Wilson, M. Goldblum (2025). Small Batch Size Training for Language Models: When Vanilla SGD Works, and Why Gradient Accumulation Is Wasteful. [arXiv:2507.07101](#)
- H. Zhang, D. Morwani, N. Vyas, J. Wu, D. Zou, U. Ghai, D. Foster, S. Kakade (2024). How Does Critical Batch Size Scale in Pre-training? [arXiv:2410.21676](#)
- M. Wortsman, P. J. Liu, L. Xiao et al. (2023). Small-scale proxies for large-scale Transformer training instabilities. [arXiv:2309.14322](#)
- C. J. Shallue, J. Lee, J. Antognini, J. Sohl-Dickstein, R. Frostig, G. E. Dahl (2018). Measuring the Effects of Data Parallelism on Neural Network Training. [arXiv:1811.03600](#)

- D. S. Kalra, M. Barkeshli (2024). Why Warmup the Learning Rate? Underlying Mechanisms and Improvements. [arXiv:2406.09405](#)